Projekt M-II
Chaotyczne przeksztalcanie obrazu cyfrowego

Dokumentacja projektu

Opracowal: Daniel Legacinski

Technologia: Python, NumPy, Pillow, Matplotlib, Tkinter

Rok: 2026

1. Cel projektu

Celem projektu jest praktyczne pokazanie mechanizmow przeksztalcania obrazu cyfrowego. Projekt nie ma tworzyc bezpiecznego szyfru, tylko eksperyment dydaktyczny pokazujacy roznice pomiedzy prosta permutacja, substytucja, pseudo-losowoscia i wizualnym chaosem.

W projekcie zostaly zrealizowane trzy obowiazkowe etapy. Kazdy etap ma osobny algorytm scramblingu oraz osobny algorytm odwrotny, czyli unscrambling. Odwracanie korzysta tylko z obrazu wynikowego oraz klucza/parametrow. Program nie zapisuje dodatkowej mapy pomocniczej.

Najwazniejsze zalozenie: jezeli klucz i parametry sa poprawne, obraz musi zostac odtworzony piksel po pikselu. Jezeli klucz jest bledny, program powinien pokazac, ze wynik odtworzenia jest niepoprawny.

2. Struktura projektu

Plik/folder	Znaczenie
run_gui.py	glowna aplikacja graficzna GUI
run_experiments.py	uruchamianie eksperymentow bez GUI i zapis wynikow
test_reversibility.py	test poprawnej odwracalnosci dla etapow 1, 2 i 3
scrambler/algorithms.py	implementacja algorytmow scramblingu i unscramblingu
scrambler/metrics.py	metryki: korelacja sasiadow, MSE, MAE, PSNR, roznice pikseli
assets/samples	obrazy testowe: obraz naturalny i obraz o silnej strukturze
outputs	wyniki eksperymentow, obrazy, CSV i wykresy

3. Reprezentacja obrazu cyfrowego

Obraz jest traktowany jako macierz pikseli RGB. Kazdy piksel ma trzy skladowe: R, G i B, a kazda skladowa jest liczba calkowita z zakresu 0-255. Po wczytaniu obraz jest konwertowany do formatu RGB, aby wszystkie algorytmy pracowaly na tej samej strukturze danych.

Dla obrazu o wysokosci H i szerokosci W liczba pikseli wynosi N = H * W. W Etapie 2 i 3 obraz jest tymczasowo zamieniany na jednowymiarowa tablice pikseli, aby mozna bylo wykonac permutacje indeksow.

```
flat = arr.reshape(H * W, 3)
# jeden element flat[i] oznacza jeden piksel RGB
```

4. Etap 1 - naiwny scrambling

Etap 1 jest celowo prosty. Program wykonuje deterministyczne przesuniecia wierszy oraz kolumn. Parametr liczbowy okresla krok przesuniecia, a klucz jest zamieniany na seed. Ten etap jest odwracalny, ale nie usuwa dobrze struktury obrazu.

Scrambling: dla kazdego wiersza wykonywane jest przesuniecie cykliczne w prawo, a dla kazdej kolumny przesuniecie cykliczne w dol. Unscrambling wykonuje te same operacje w odwrotnej kolejnosci i z przeciwnym znakiem przesuniecia.

```
for r in range(H):
    row[r] = roll(row[r], shift_r)
for c in range(W):
    column[c] = roll(column[c], shift_c)
# odwrotnie: najpierw kolumny w gore, potem wiersze w lewo
```

Slabosc metody: tekst, szachownica, gradienty i krawedzie moga nadal byc czesciowo widoczne, poniewaz piksele w lokalnych strukturach sa tylko przesuwane, a nie mieszane globalnie.

5. Etap 2 - czysta permutacja sterowana kluczem

Etap 2 realizuje czysta permutacje pikseli. Wartosc piksela nie jest zmieniana, zmienia sie tylko jego pozycja. Permutacja jest generowana algorytmem Fishera-Yatesa sterowanym jawnym generatorem LCG. Dzięki temu dla tego samego klucza i parametru zawsze powstaje ta sama permutacja.

Formalnie permutacja jest funkcja P: {0,...,N-1} -> {0,...,N-1}. Algorytm odwrotny korzysta z tej samej permutacji i rozklada piksele z powrotem na poprzednie indeksy.

```
scrambled[i] = original[P[i]]
restored[P[i]] = scrambled[i]
# dlatego P^(-1)(P(i)) = i
```

Czysta permutacja zmniejsza widocznosc lokalnych struktur, ale sama nie jest bezpieczna. Histogram kolorow pozostaje identyczny, bo wartosci pikseli nie sa zmieniane.

6. Etap 3 - mechanizm wzmacniajacy

W Etapie 3 zastosowano klase C: hybryda, czyli permutacja + substytucja. Najpierw obraz jest permutowany jak w Etapie 2, a potem do kazdej skladowej RGB dodawany jest strumien wartosci 0-255 modulo 256. Strumien jest generowany deterministycznie z klucza, parametru i numeru rundy.

Operacja substytucji jest odwracalna, poniewaz dodawanie modulo 256 mozna cofnac odejmowaniem modulo 256. Przy odtwarzaniu trzeba wykonac operacje w odwrotnej kolejnosci: najpierw odjac strumien, potem cofnac permutacje.

```
scramble:   y = (P(x) + stream) mod 256
unscramble: x = P^(-1)((y - stream) mod 256)
```

Mechanizm poprawia efekt wizualny, bo zmienia nie tylko polozenie pikseli, ale tez ich wartosci. Nadal nie jest to bezpieczny szyfr, poniewaz generator LCG i funkcja seed sa proste oraz mozliwe do analizy.

7. Interfejs uzytkownika GUI

Aplikacja GUI zostala wykonana w Tkinter. Interfejs pozwala wczytac obraz PNG, JPEG lub BMP, wybrac etap 1/2/3, wpisac klucz, podac parametr i liczbe rund dla Etapu 3. Program jednoczesnie pokazuje obraz oryginalny, obraz przeksztalcony oraz obraz odtworzony.

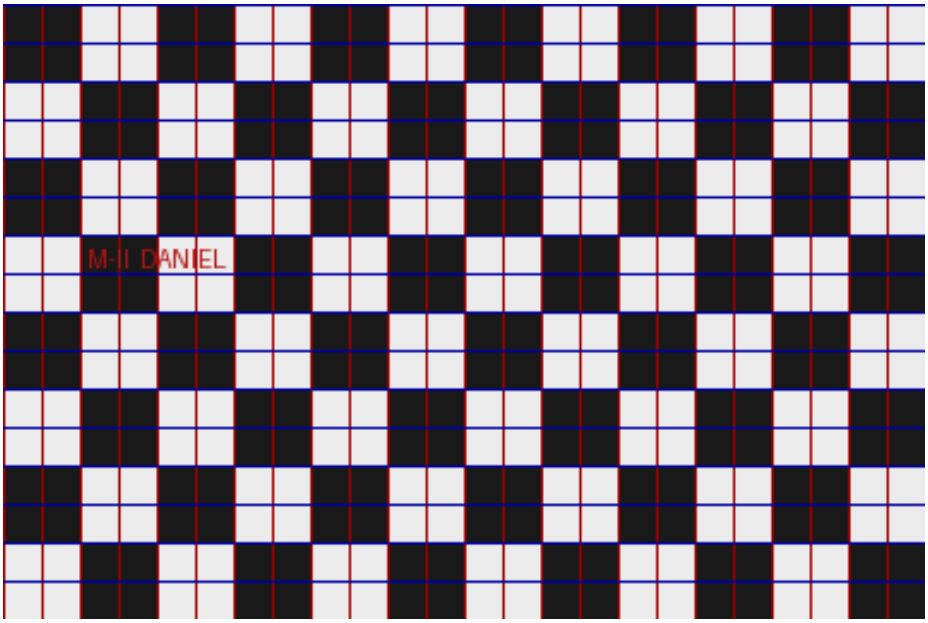
Dostepne przyciski: Wczytaj obraz, Scramble, Unscramble - poprawny klucz, Unscramble - bledny klucz, Zapisz aktualne wyniki oraz Uruchom pelne eksperymenty. GUI jest narzedziem analitycznym, a nie tylko dekoracja.

8. Metryki eksperymentalne

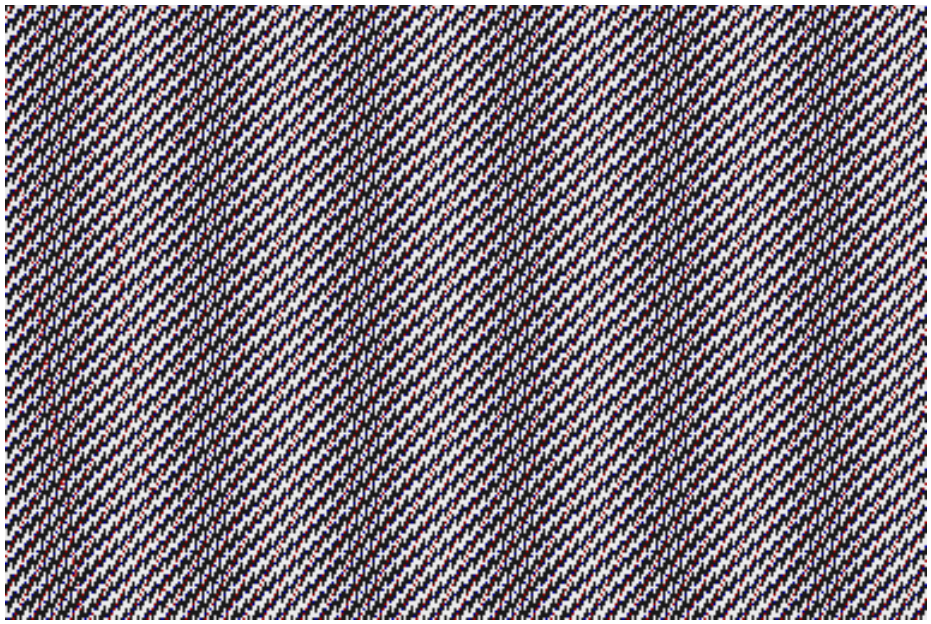
Projekt liczy dwie obowiazkowe grupy metryk. Pierwsza to korelacja sasiadujacych pikseli przed i po scramblingu. Druga to roznica obrazu po unscramblingu blednym kluczem. Dodatkowo program liczy MSE, MAE, maksymalna roznice oraz PSNR.

Metryka	Interpretacja
Korelacja sasiadow	pokazuje, czy piksele obok siebie sa nadal podobne
MSE	sredni kwadrat roznicy miedzy obrazami
MAE	srednia bezwzgledna roznica pikseli
Max diff	najwieksza roznica dla pojedynczej skladowej
PSNR	miara podobienstwa obrazow; nieskonczona dla identycznych obrazow

9. Wyniki eksperymentow

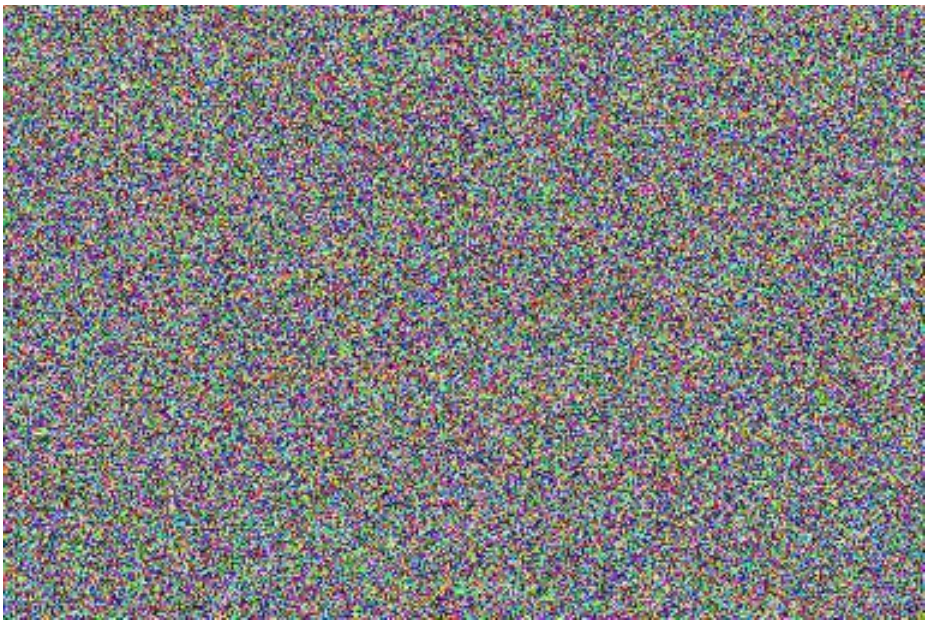


Rys. 1. Obraz testowy o silnej strukturze.

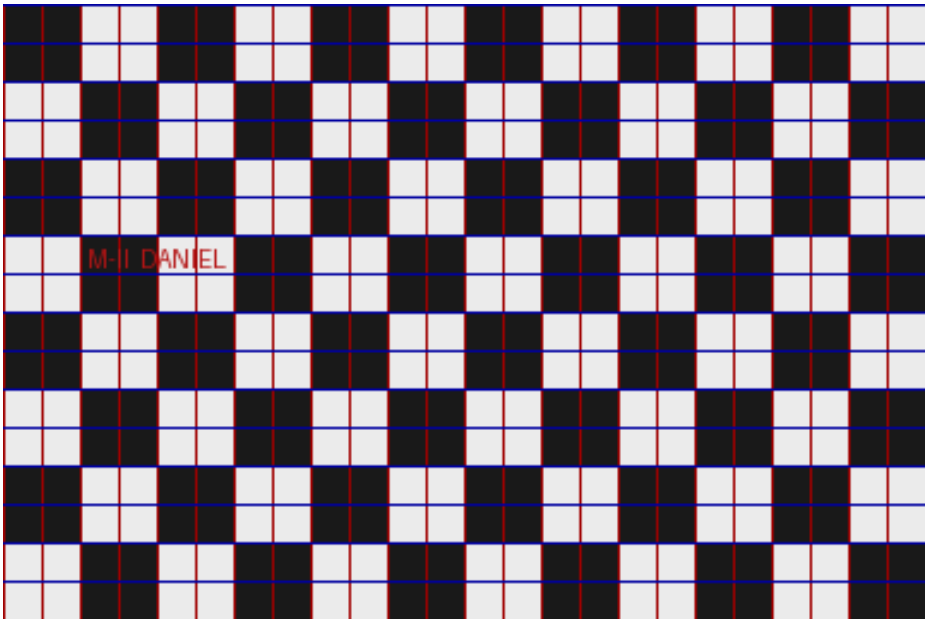


Rys. 2. Etap 1 - widoczne pozostalosci struktury po naiwnym przesunieciu.

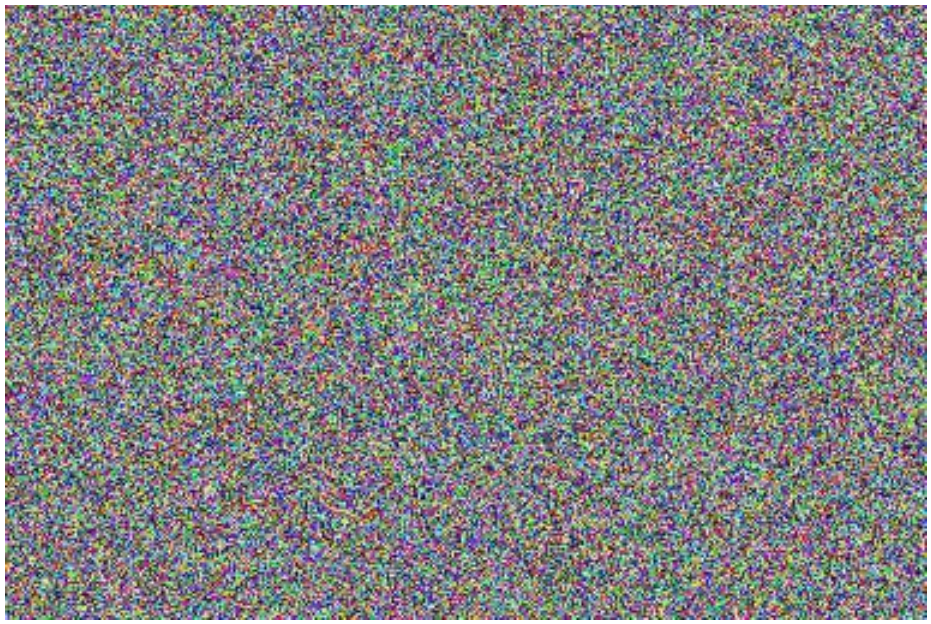
Rys. 3. Etap 2 - czysta permutacja pikseli.



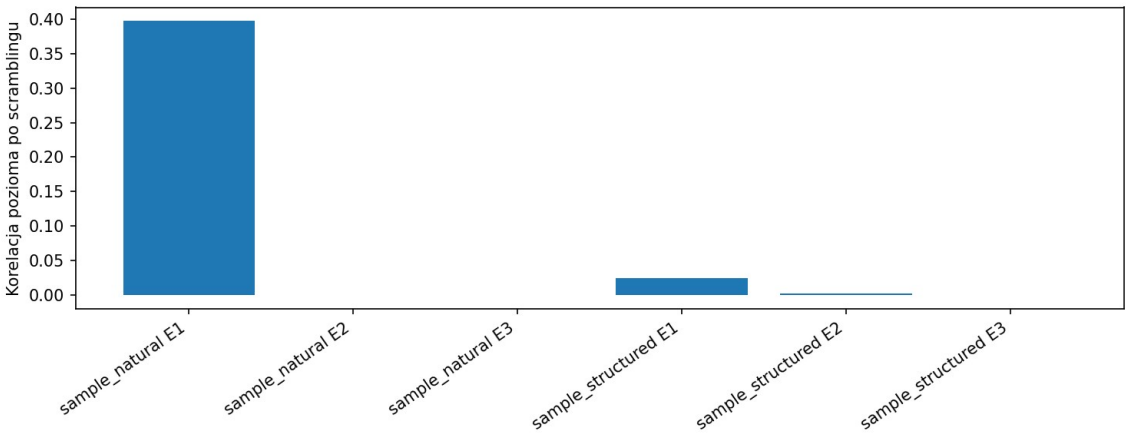
Rys. 4. Etap 3 - permutacja + substytucja modularna.



Rys. 5. Poprawne odtworzenie obrazu dla Etapu 3.



Rys. 6. Proba odtworzenia Etapu 3 blednym kluczem.



Rys. 7. Porownanie korelacji po scramblingu.

Tabela ponizej zawiera skrot wynikow z automatycznego eksperymentu. Pelny plik CSV znajduje sie w folderze outputs.

Obraz: sample_natural, etap 1 | korelacja po H: 0.3977 | odtworzenie OK: True | MSE bledny klucz: 6189.80 | MAE bledny klucz: 66.48

Obraz: sample_natural, etap 2 | korelacja po H: -0.0009 | odtworzenie OK: True | MSE bledny klucz: 6261.11 | MAE bledny klucz: 64.14

Obraz: sample_natural, etap 3 | korelacja po H: -0.0004 | odtworzenie OK: True | MSE bledny klucz: 8710.09 | MAE bledny klucz: 76.67

Obraz: sample_structured, etap 1 | korelacja po H: 0.0237 | odtworzenie OK: True | MSE bledny klucz: 23998.12 | MAE bledny klucz: 117.74

Obraz: sample_structured, etap 2 | korelacja po H: 0.0023 | odtworzenie OK: True | MSE bledny klucz: 21842.35 | MAE bledny klucz: 107.30

Obraz: sample_structured, etap 3 | korelacja po H: -0.0008 | odtworzenie OK: True | MSE bledny klucz: 16495.99 | MAE bledny klucz: 107.12

10. Analiza porownawcza etapow

Etap 1 jest najprostszy i pokazuje porazke metody naiwnej. Obraz wyglada inaczej, ale struktura moze nadal przetrwac. Przy obrazie z tekstem lub szachownica mozna rozpoznac regularnosc, poniewaz algorytm przesuwa cale wiersze i kolumny.

Etap 2 usuwa wiele lokalnych struktur, poniewaz piksele trafiaja w dalekie miejsca. Jednak wartosci pikseli pozostaja takie same, dlatego histogram obrazu i ogolna paleta kolorow sie nie zmieniaja. To oznacza, ze sama permutacja nie zapewnia bezpieczenstwa.

Etap 3 daje najsilniejszy efekt wizualny, bo laczy zmiane pozycji i zmiane wartosci pikseli. Bledny klucz powoduje duze roznice w obrazie odtworzonym. Mimo tego nie mozna twierdzic, ze jest to szyfr bezpieczny, bo uzyty generator jest prosty i przewidywalny.

11. Krytyka wlasnego rozwiazania: dlaczego to NIE jest bezpieczny szyfr

1. Jakie informacje atakujacy nadal moze odzyskac? W Etapie 1 atakujacy moze zobaczyc resztki struktury, bo przesuniecia zachowuja lokalne zaleznosci. W Etapie 2 atakujacy widzi histogram kolorow, poniewaz wartosci pikseli sie nie zmieniaja. W Etapie 3 histogram jest zmieniony, ale generator LCG jest prosty i mozliwy do odtworzenia przy analizie wielu danych.

2. Co dzieje sie przy ataku typu known-plaintext? Jezeli atakujacy zna obraz oryginalny i wynikowy dla tego samego klucza, moze probowac odzyskac permutacje albo parametry generatora. W Etapie 2 jest to szczegolnie grozne, bo permutacja bezposrednio laczy piksele oryginalne z wynikowymi. W Etapie 3 atak jest trudniejszy, ale nadal mozliwy, bo substytucja jest prosta i deterministyczna.

3. Dlaczego zwiekszanie chaosu nie rozwiazuje problemu? Obraz moze wygladac losowo, ale wyglad losowy nie oznacza bezpieczenstwa. Bezpieczenstwo wymaga odpornego projektu kryptograficznego, analizy klucza, odpornego generatora i testow kryptograficznych. Ten projekt celowo tego nie zapewnia, poniewaz ma charakter dydaktyczny.

12. Instrukcja uruchomienia

Krok 1. Rozpakuj folder projektu. Krok 2. Otworz terminal w tym folderze. Krok 3. Zainstaluj biblioteki poleceniem pip install -r requirements.txt. Krok 4. Uruchom aplikacje poleceniem python run_gui.py.

```
pip install -r requirements.txt
python run_gui.py
```

Aby wykonac eksperymenty automatycznie bez GUI, uruchom:

```
python run_experiments.py
```

Aby sprawdzic odwracalnosc wszystkich etapow, uruchom:

```
python test_reversibility.py
```

13. Zalacznik techniczny - pseudokod algorytmow

Pseudokod ponizej pokazuje, ze kazdy etap ma osobna procedure odwrotna. Jest to wazne, poniewaz wymaganie projektu zabrania przechowywania dodatkowych danych pomocniczych. Program musi odtworzyc obraz tylko z obrazu wynikowego oraz klucza/parametrow.

```
ETAP 1 SCRAMBLE:
  dla kazdego wiersza r: przesun wiersz o shift_r
  dla kazdej kolumny c: przesun kolumne o shift_c
ETAP 1 UNSCRAMBLE:
  dla kazdej kolumny c od konca: przesun kolumne o -shift_c
  dla kazdego wiersza r od konca: przesun wiersz o -shift_r

ETAP 2 SCRAMBLE:
  seed = funkcja(klucz, parametr)
  P = FisherYates(N, seed)
  wynik[i] = obraz[P[i]]
ETAP 2 UNSCRAMBLE:
  P = FisherYates(N, seed)
  odtworzony[P[i]] = wynik[i]

ETAP 3 SCRAMBLE:
  dla kazdej rundy:
    obraz = permutacja(obraz)
    obraz = (obraz + strumien) mod 256
ETAP 3 UNSCRAMBLE:
  dla kazdej rundy w odwrotnej kolejnosci:
    obraz = (obraz - strumien) mod 256
    obraz = odwrotna_permutacja(obraz)
```

Wazna obserwacja: odwracalnosc nie oznacza bezpieczenstwa. Odwracalnosc jest tylko warunkiem technicznym, aby poprawny uzytkownik mogl odzyskac obraz. Bezpieczenstwo wymagaloby odpornosci na analize atakujacego, czego projekt celowo nie zapewnia.

14. Wnioski

Projekt pokazuje, ze przeksztalcanie obrazu moze byc w pelni odwracalne bez przechowywania danych pomocniczych. Najwazniejszy warunek to jednoznaczny klucz, parametr i algorytm odwrotny. Etap 1 pokazuje slabosc metod naiwnych, Etap 2 pokazuje ograniczenia samej permutacji, a Etap 3 pokazuje, ze dodanie substytucji wzmacnia efekt wizualny.

Najwazniejszy wniosek jest taki, ze wizualny chaos nie jest tym samym co bezpieczenstwo. Nawet jezeli obraz wyglada jak przypadkowy szum, algorytm moze byc latwy do przeanalizowania. Dlatego projekt nalezy traktowac jako eksperyment edukacyjny, a nie jako narzedzie do ochrony danych.

15. Bibliografia i zrodla

Dokumentacja Python: https://docs.python.org/3/

Dokumentacja NumPy: https://numpy.org/doc/

Dokumentacja Pillow: https://pillow.readthedocs.io/

Dokumentacja Matplotlib: https://matplotlib.org/stable/

Opis permutacji Fisher-Yates: https://en.wikipedia.org/wiki/Fisher%E2%80%93Yates_shuffle

Opis liniowego generatora kongruencyjnego LCG:
https://en.wikipedia.org/wiki/Linear_congruential_generator